REST Services with JAX-RS 2 and Eclipse

2023/2024

Diego Sevilla Ruiz

DITEC
Facultad de Informática

Murcia, 2026

Contents

1. Introduction
2. Eclipse Project
3. Visual Studio Code
4. Maven & Docker
5. JAX-RS (Jersey)

Contents

1. Introduction
2. Eclipse Project
3. Visual Studio Code
4. Maven & Docker
5. JAX-RS (Jersey)

Introduction

In this practice session:

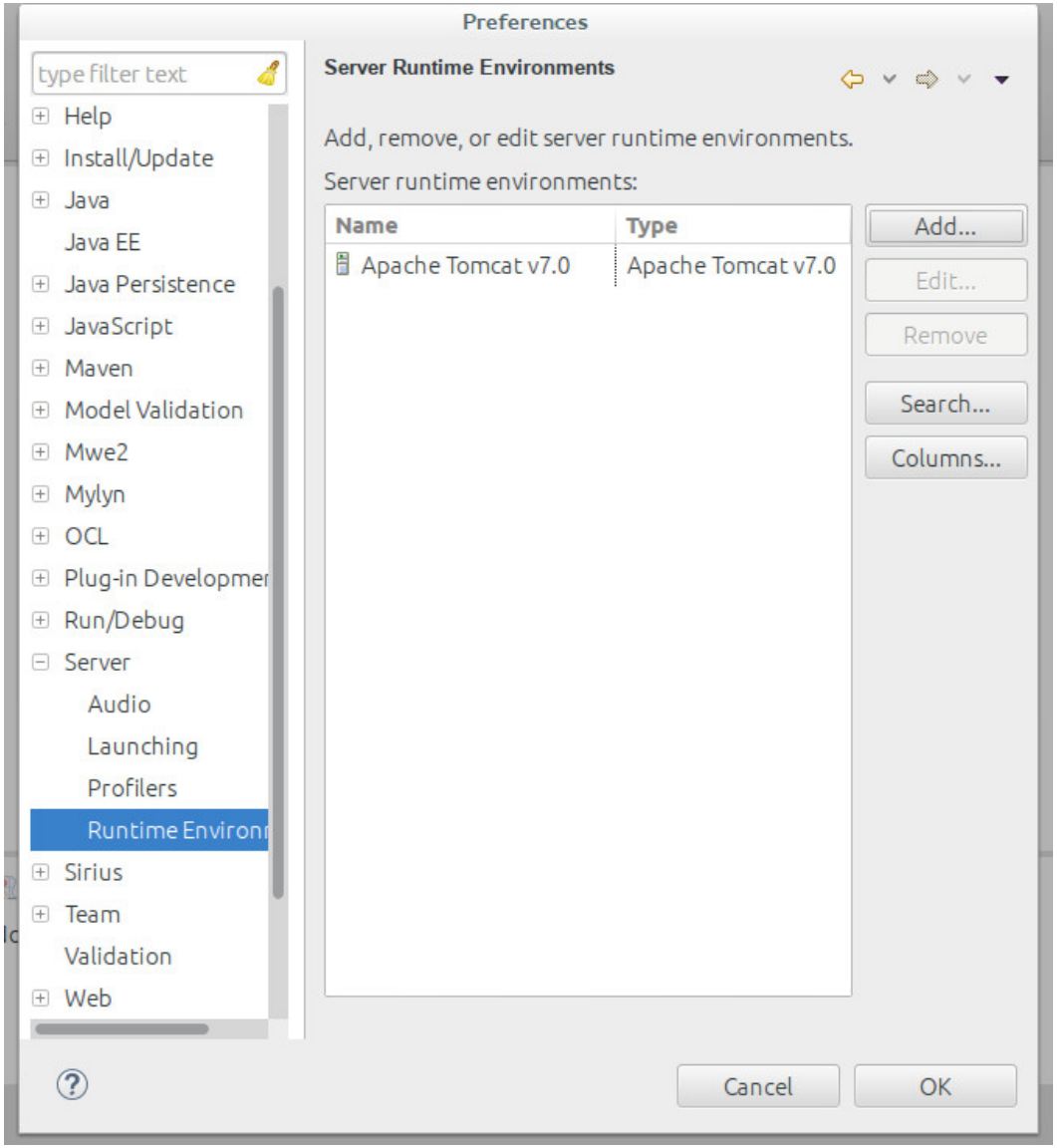
1. Will download and configure Eclipse (in Linux) to run REST projects (JAX-RS/Jersey)
2. We will create a dynamic Web project
3. Configure it to support Jersey (JAX-RS implementation)
4. We will study the basics of JAX-RS

Downloading the Software

- In this case Linux:
  - Download Tomcat 10.1
    `http://tomcat.apache.org/download-10.cgi` (`.tar.gz` or `.zip`)
  - Eclipse IDE for JavaEE
    `http://www.eclipse.org/downloads/`

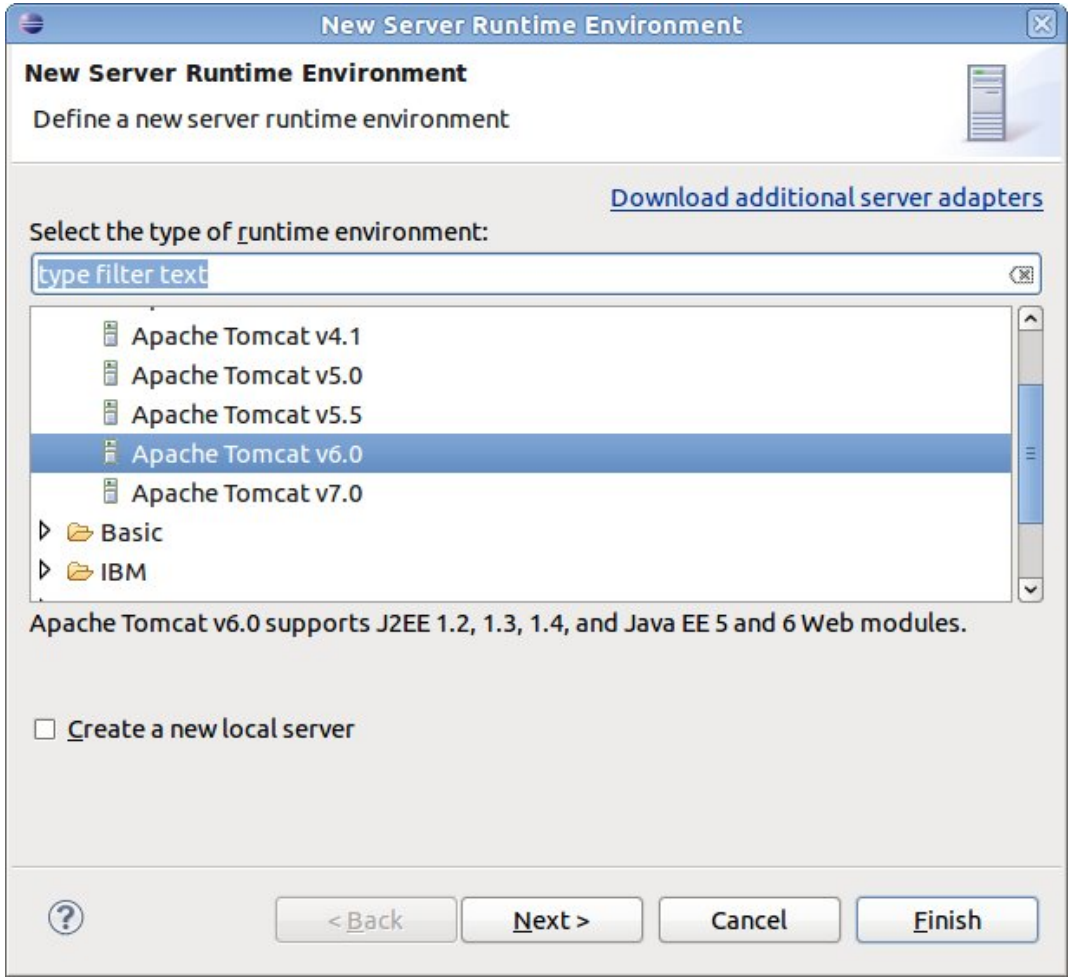
Eclipse Configuration – Server Environment

Window → Preferences → Server → Runtime Environment



Eclipse Configuration – Server Environment (ii)

Add... → Apache Tomcat 10.0 → Next...



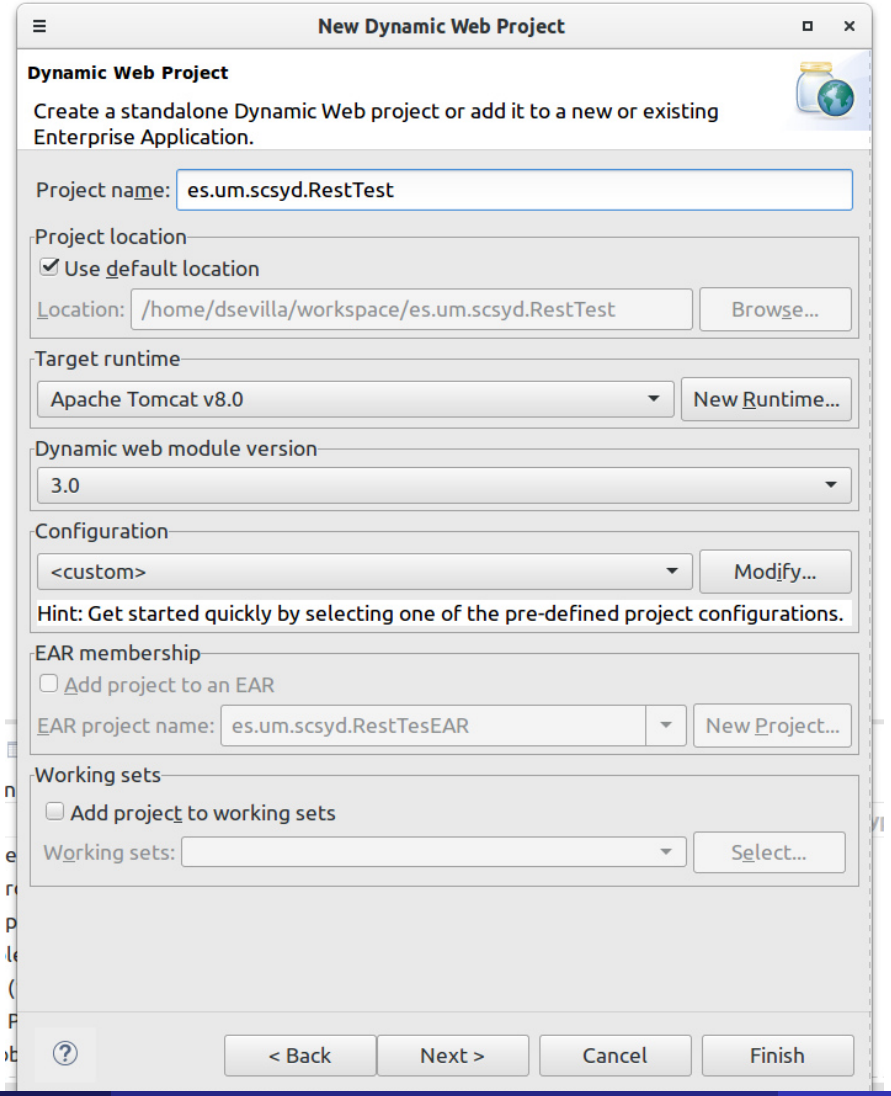
Add the directory where the Tomcat server is

Contents

1. Introduction
2. Eclipse Project
3. Visual Studio Code
4. Maven & Docker
5. JAX-RS (Jersey)

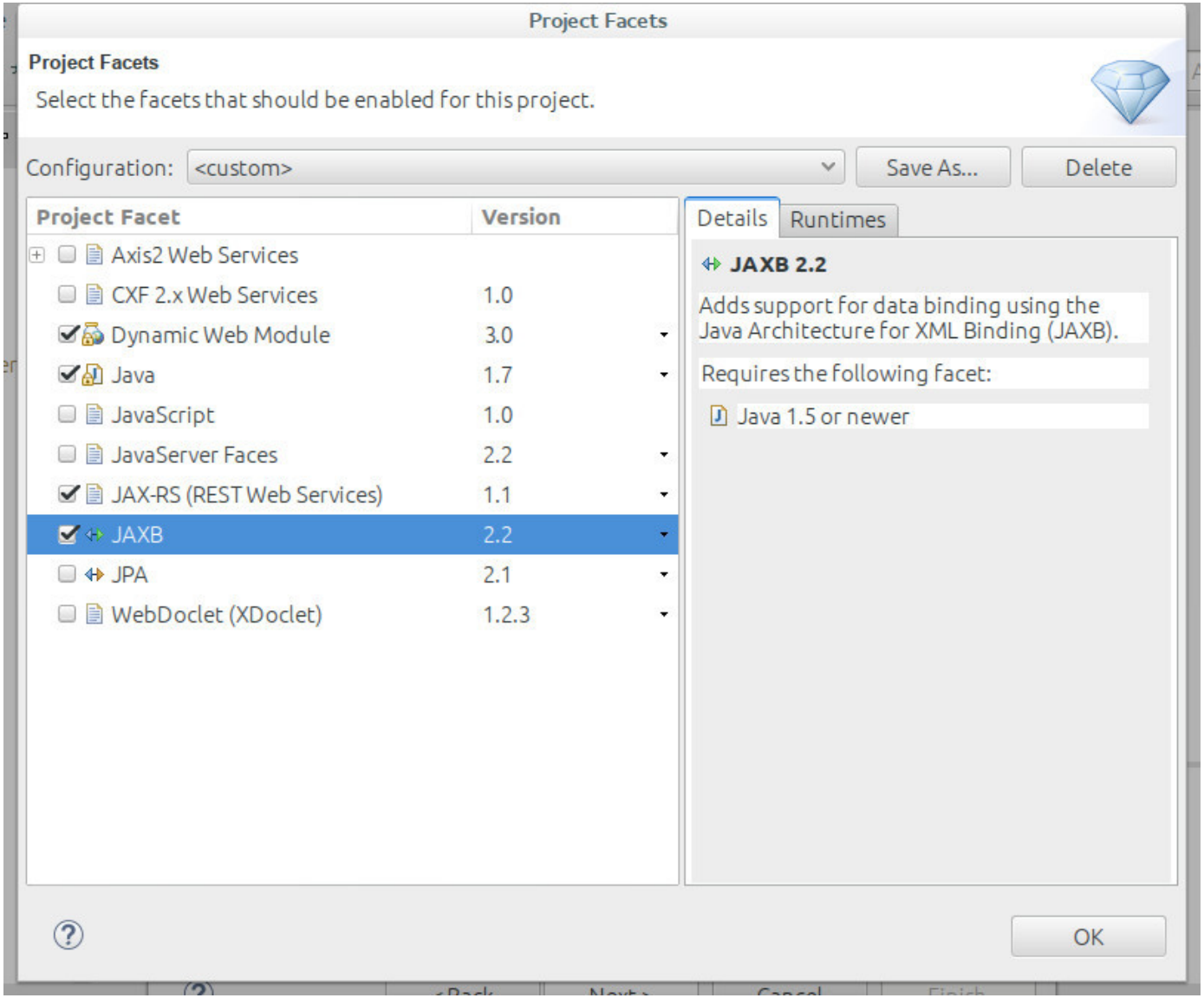
Eclipse Project

File → New → Project... → Web → Dynamic Web Project
(Dynamic Web Module 4 or 5)



Eclipse Project (ii)

Project properties → Project Facets... → Modify... (JAXB 2.x & JAX-RS 2.x)



Eclipse Project (iv)

Configure → Convert to Maven Project

Java EE – es.um.scsyd.RestTst/pom.xml – Eclipse

File Edit Navigate Search Project Run Window Help

Quick Access | Java | Debug | Java EE

Project Explorer

- es.um.scsyd.RestTst
  - JAX-WS Web Services
  - Deployment Descriptor: es.um.scsyd.Re
  - JAXB Content
  - Java Resources
    - src
    - Libraries
  - JavaScript Resources
  - build
  - target
  - WebContent
    - META-INF
      - MANIFEST.MF
    - WEB-INF
      - lib
      - web.xml
  - pom.xml

es.um.scsyd.RestTst/pom.xml | web.xml

Dependencies

Filter:

Dependencies: Add... | Remove | Properties... | Manage...

Dependency Management: Add... | Remove | Properties...

To manage your transitive dependency exclusions, please use the Dependency Hierarchy page.

Overview | Dependencies | Dependency Hierarchy | Effective POM | pom.xml

Outline | Task List

project xmlns=http://maven.apache.or

Markers | Properties | Servers | Data Source Explorer | Snippets

0 errors, 2 warnings, 12 others

Description	Resource	Path	Location	Type
Build path specifies execution environment	json2nosqlV1		Build path	JRE System Library Problem
Java Line Breakpoints (1 item)				
Java Task (11 items)				

es.um.scsyd.RestTst

Eclipse Project (vii)

pom.xml:

```
(muy grande para listarlo)
BOM de jakarta-ee
BOM de glassfish-jersey
BOM de glassfish-jaxb
```

Eclipse Project – Java Class

In the subdirectory `Java Resources:src`, create the file `HelloApp.java`:

```
package es.um.sisdist.RestTest.first;

import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.core.Application;

@ApplicationPath("/jaxrs")
public class HelloApp extends Application
{
    // Empty on purpose
}
```

- The class must have the `@ApplicationPath` annotation
- This will mark the base of all the resource ("`/jaxrs`")
- All classes marked with `@Path` will be under this route

Eclipse Project – Resource Java Class

In the subdirectory `Java Resources:src`, create the file `Hello.java`

```
package es.um.sisdist.RestTest.first;

import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;
```

Eclipse Project – Resource Java Class (ii)

POJO: *Plain Old Java Object*, do not implement interfaces, no `extends`

```
// POJO, no interface no extends

// Sets the path to base URL + /hello
@Path("/hello")
public class Hello
{
    // This method is called if TEXT_PLAIN is requested
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String sayPlainTextHello() {
        return "Hello Jersey";
    }
```

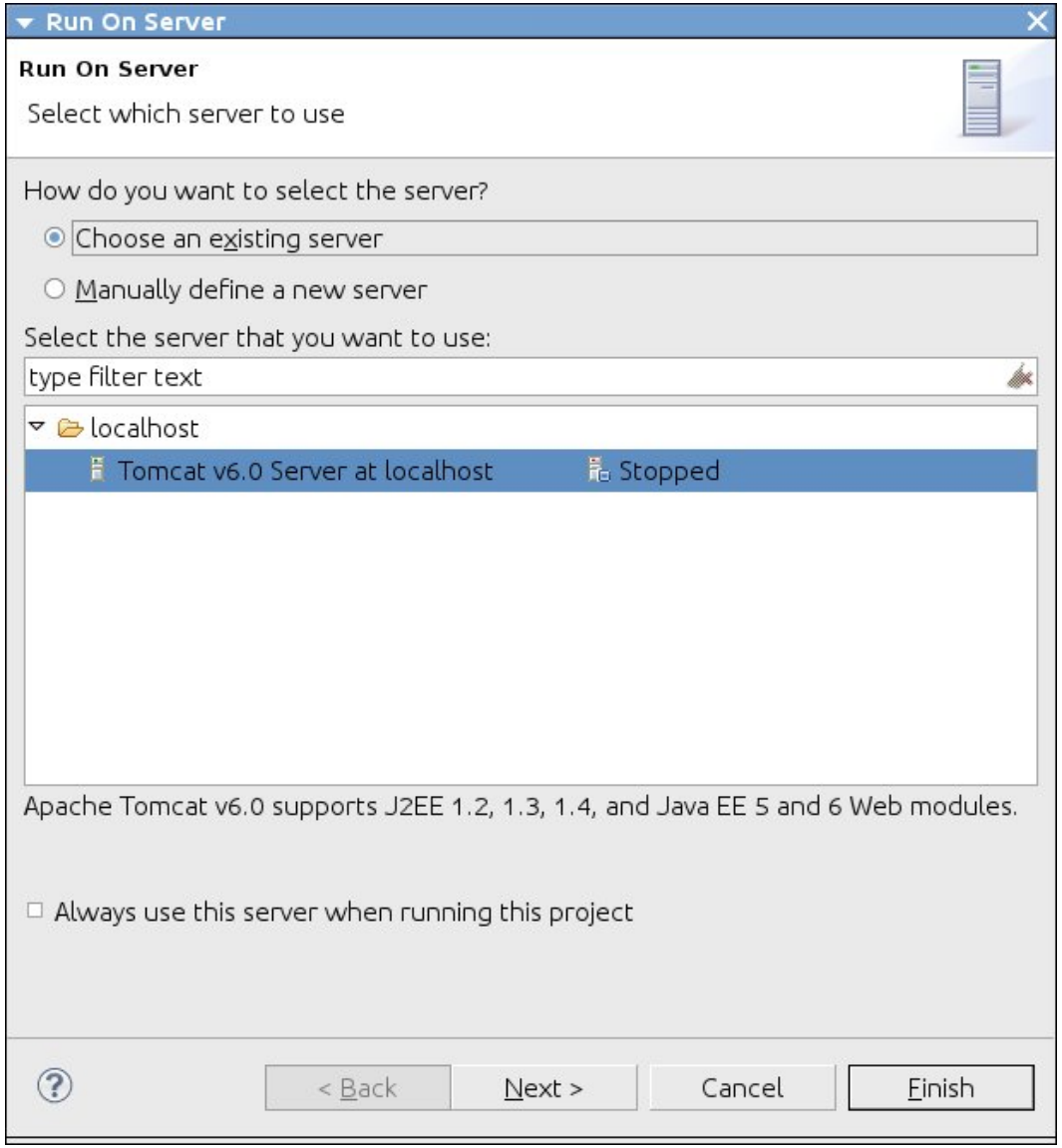
- Annotations
  - `@Path`
  - `@GET`
  - `@Produces`

Eclipse Project – Resource Java Class (iii)

```
// This method is called if XML is requested
@GET
@Produces(MediaType.TEXT_XML)
public String sayXMLHello() {
    return "<?xml version=\"1.0\"?>" + "<hello> Hello Jersey" + "</hello>";
}

// This method is called if HTML is requested
@GET
@Produces(MediaType.TEXT_HTML)
public String sayHtmlHello() {
    return "<html> " + "<title>" + "Hello Jersey" + "</title>"
        + "<body><h1>" + "Hello Jersey" + "</h1></body>" + "</html>";
}
```

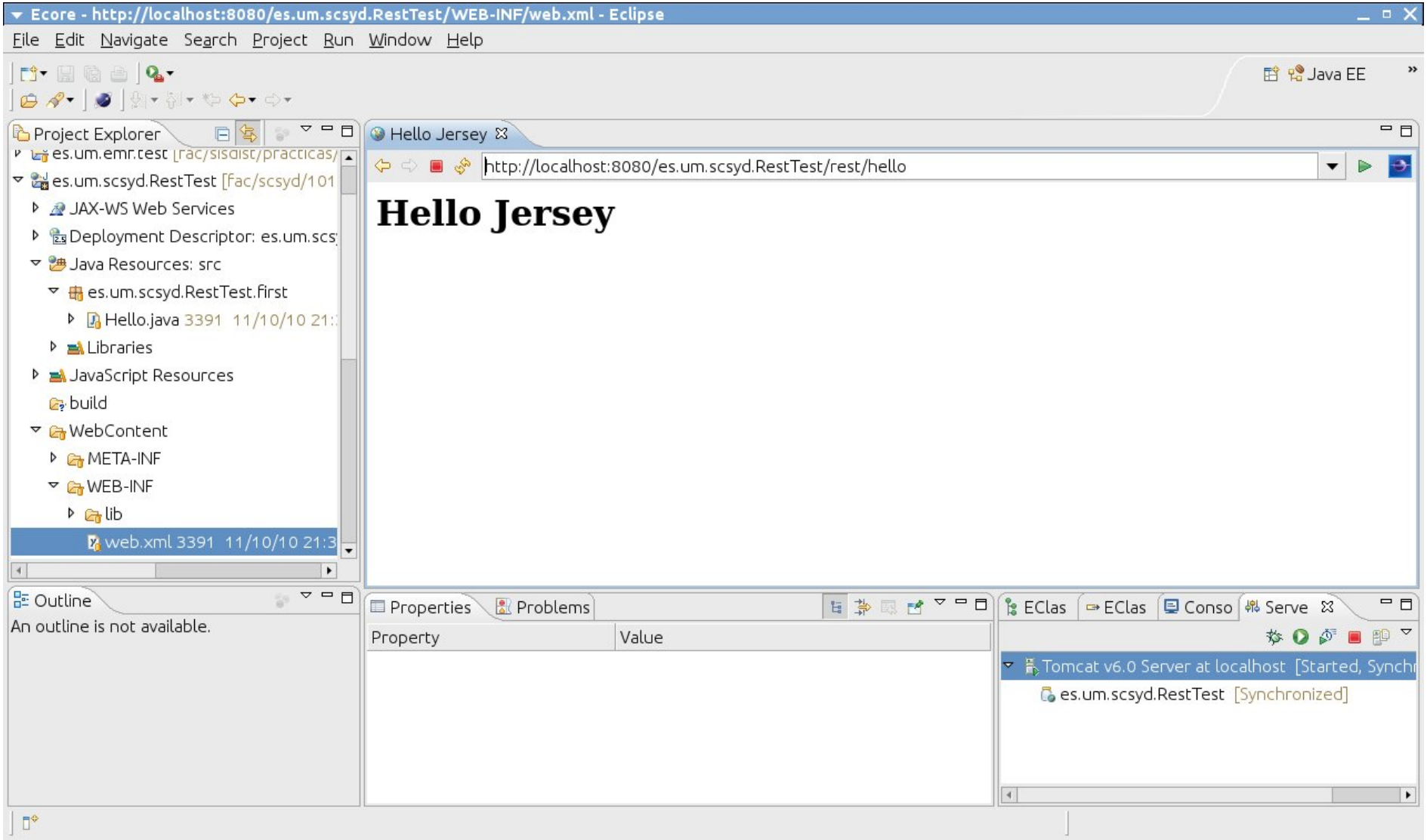
Run the service

To run the service: Project root → Run As... → Run on Server



Run the service (ii)

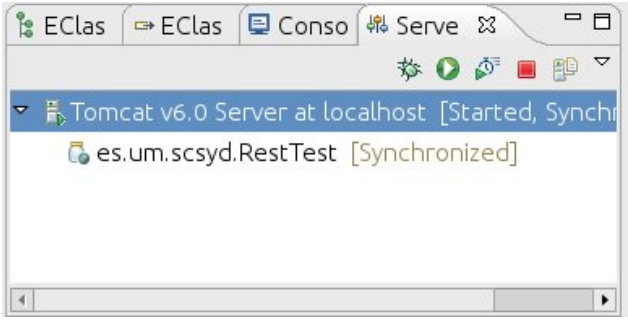
Service running (JavaEE perspective)



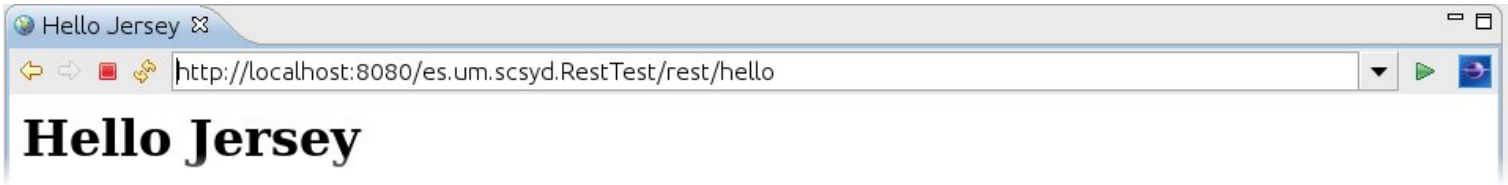
Run the service (iii)

In JavaEE perspective:

- The lower right corner shows the server running



- http://localhost:8080/es.um.sisdist.RestTest/jaxrs/hello



  - The server is started on port 8080
  - Context, the name of the project (*es.um.sisdist.RestTest*)
  - The service base (*jaxrs/**)
  - The `@Path` of the Java file (*/hello*)

Eclipse Project – Client

In the subdirectory `Java Resources: src`, create the file `TestClient.java`

```
package es.um.sisdist.RestTest.first;

import java.net.URI;

import org.glassfish.jersey.client.ClientConfig;

import com.fasterxml.jackson.core.util.JacksonFeature;

import jakarta.ws.rs.client.Client;
import jakarta.ws.rs.client.ClientBuilder;
import jakarta.ws.rs.client.WebTarget;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.UriBuilder;
```

Eclipse Project – Client (ii)

```
1   public class TestClient
    {
3   public static void main(String[] args) {
        ClientConfig config = new ClientConfig();
5       ClientBuilder.newBuilder().register(JacksonFeature.class);
        Client client = ClientBuilder.newClient(config);
7       WebTarget service = client.target(getBaseURI());
        // Fluent interfaces
9       System.out.println(service.path("jaxrs").path("hello").request(
                MediaType.TEXT_PLAIN).get(String.class).toString());
11      // Get plain text
        System.out.println(service.path("jaxrs").path("hello").request(
13              MediaType.TEXT_PLAIN).get(String.class));
        // Get XML
15      System.out.println(service.path("jaxrs").path("hello").request(
                MediaType.TEXT_XML).get(String.class));
17      // The HTML
        System.out.println(service.path("jaxrs").path("hello").request(
19              MediaType.TEXT_HTML).get(String.class));
    }
21
    private static URI getBaseURI() {
23      return UriBuilder.fromUri(
                "http://localhost:8080/es.um.sisdist.RestTest").build();
25  }
    }
```

`TestClient.java` (right click) → Run As... → Java Application

Eclipse Project – Client (iii)

Output

EClass Hierarchy | EClass References | Console | Servers

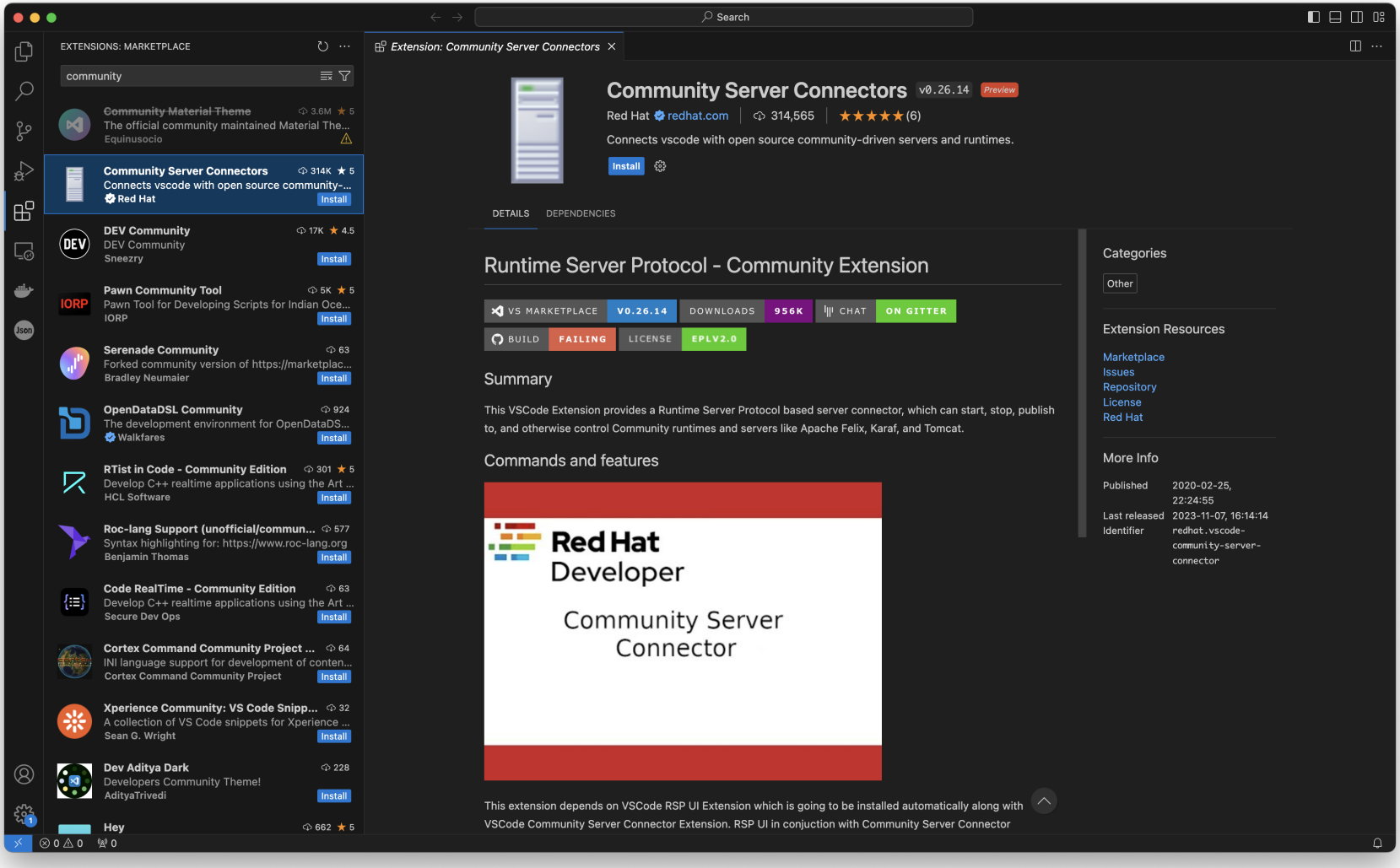
```
<terminated> TestClient [Java Application] /usr/lib/jvm/java-6-sun-1.6.0.22/bin/java (02/11/2010 13:25:43)
GET http://localhost:8080/es.um.scsyd.RestTest/rest/hello returned a response status of 200
Hello Jersey
<?xml version="1.0"?><hello> Hello Jersey</hello>
<html> <title>Hello Jersey</title><body><h1>Hello Jersey</body></h1></html>
```

Contents

1. Introduction
2. Eclipse Project
3. Visual Studio Code
4. Maven & Docker
5. JAX-RS (Jersey)

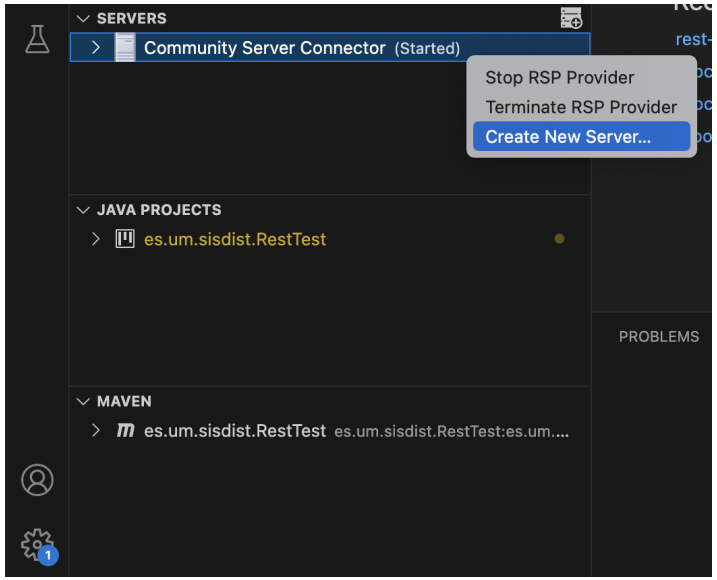
Install Tomcat

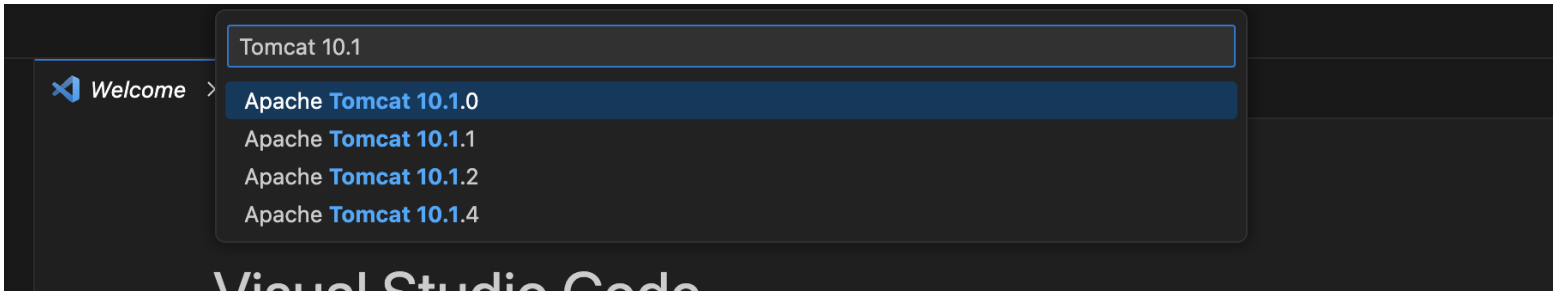
Extensions → Community Server Connectors



Running Tomcat

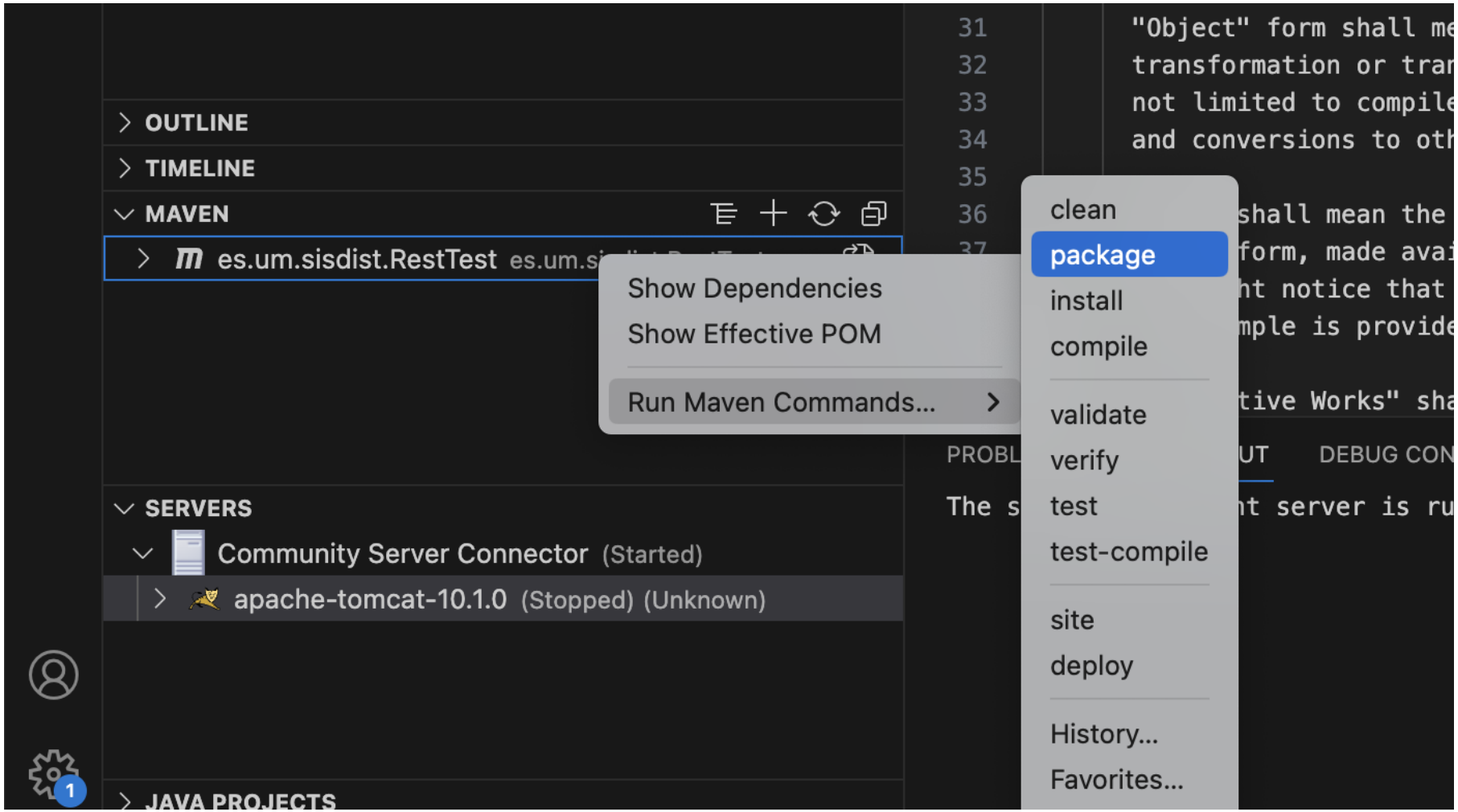
Community Server Connector → Create New Server → Apache Tomcat X.X.X





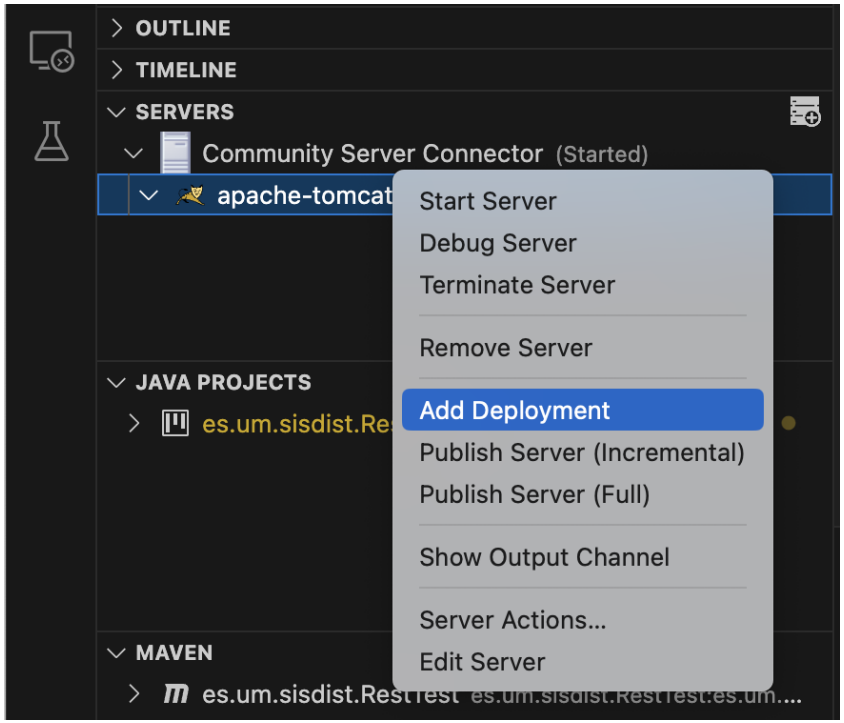
Building the .WAR

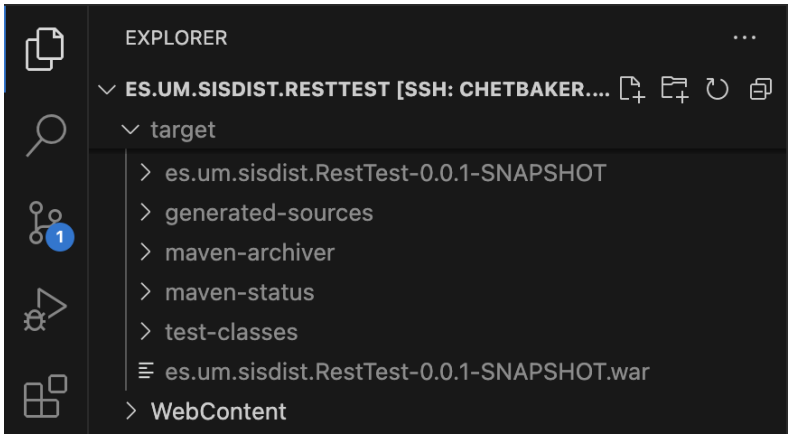
Maven → Run Maven Commands → package



Running the .WAR on Tomcat

Community Server Connector → Add Deployment →
→ es.*.SNAPSHOT.war → Publish Server (Full)





Troubleshooting

- Server not running?
  - Publish Server (Full)
  - Stop/Start Sever
- `http://localhost:8080/es.um.sisdist.RestTest-0.0.1-SNAPSHOT/jaxrs/hello`

Contents

1. Introduction
2. Eclipse Project
3. Visual Studio Code
4. Maven & Docker
5. JAX-RS (Jersey)

Maven & Docker

- mvn package
- docker-compose up
- `http://localhost:8080/Service/jaxrs/hello`

Contents

1. Introduction
2. Eclipse Project
3. Visual Studio Code
4. Maven & Docker
5. JAX-RS (Jersey)

JAX-RS (Jersey)

- JAX-RS is a specification from Sun (Oracle) to implement REST services using Java classes + annotations
- Defined in specification document JSR-339[1] (JSR311 for JAX-RS 1.x)
- Eclipse Jersey[2] is the reference implementation, though there are others, such as Apache CXF

[1] `https://jcp.org/en/jsr/detail?id=339`.
[2] `https://eclipse-ee4j.github.io/jersey/`.

Classes as Resources

- Based on the concept of POJO (*Plain Old Java Object*)
  - Classes without `extends` or `implements` (Martin Fowler)
  - Term inherited from C/C++: *Plain Old Data* (POD)
  - Makes sense, they are data classes
- There must be a root resource class
  - It also makes sense, since the URIs are hierarchical
  - There has to be an annotation `@Path` in the class or a method
    - ⇒ Specifies the initial *path*
  - Methods annotated `@GET`, `@PUT`, etc.

Summary of Annotations

Annotation	Description
`@Path`	The `@Path` annotation's value is a relative URI path indicating where the Java class will be hosted, for example, `/helloworld`. You can also embed variables in the URIs to make a URI path template. For example, you could ask for the name of a user, and pass it to the application as a variable in the URI, like this, `/helloworld/{username}`. `@Path` annotations in operations are made relative to the `@Path` annotation in their containing class.
`@GET`	The `@GET` annotation is a request method designator and corresponds to the similarly named HTTP method. The Java method annotated with this request method designator will process HTTP GET requests.
`@POST`	The `@POST` annotation is a request method designator and corresponds to the similarly named HTTP method. The Java method annotated with this request method designator will process HTTP POST requests.
`@PUT`	The `@PUT` annotation is a request method designator and corresponds to the similarly named HTTP method. The Java method annotated with this request method designator will process HTTP PUT requests.
`@DELETE`	The `@DELETE` annotation is a request method designator and corresponds to the similarly named HTTP method. The Java method annotated with this request method designator will process HTTP DELETE requests.

Summary of Annotations (ii)

Annotation	Description
`@HEAD`	The `@HEAD` annotation is a request method designator and corresponds to the similarly named HTTP method. The Java method annotated with this request method designator will process HTTP HEAD requests.
`@PathParam`	The `@PathParam` annotation is a type of parameter that you can extract for use in your resource class. URI path parameters are extracted from the request URI, and the parameter names correspond to the URI path template variable names specified in the `@Path` class-level annotation.
`@QueryParam`	The `@QueryParam` annotation is a type of parameter that you can extract for use in your resource class. Query parameters are extracted from the request URI query parameters.
`@Consumes`	The `@Consumes` annotation is used to specify the MIME media types of representations a resource can consume that were sent by the client.
`@Produces`	The `@Produces` annotation is used to specify the MIME media types of representations a resource can produce and send back to the client, for example, "`text/plain`".

Summary of Annotations (iii)

Annotation	Description
`@Provider`	The `@Provider` annotation is used for anything that is of interest to the JAX-RS runtime, such as `MessageBodyReader` and `MessageBodyWriter`. For HTTP requests, the `MessageBodyReader` is used to map an HTTP request entity body to method parameters. On the response side, a return value is mapped to an HTTP response entity body using a `MessageBodyWriter`. If the application needs to supply additional metadata, such as HTTP headers or a different status code, a method can return a `Response` that wraps the entity, and which can be built using `Response.ResponseBuilder`.

URL parameters

```
@Path("/users/{username}")
public class UserResource {
    @GET
    @Produces("text/xml")
    public String getUser(@PathParam("username")
        String userName) {
    ...
    }
}
```

- The pattern `{username}` implies a variable (parameter) of the URL
- Through the `@PathParam` annotation a URL parameter is associated with a parameter of the method
- The default regular expression for the parameter is «`[^/]+?`», but you can specify another one:

```
@Path("users/{username: [a-zA-Z][a-zA-Z_0-9]}")
```

Methods implemented by default

- `@HEAD`
  - Automatically implemented by calling the method labelled with `@GET`, and ignoring the response
- `@OPTIONS`
  - Returns in the `Allow` header the set of methods that the resource supports
  - Jersey would also return a WADL description (*Web Application Description Language*)[3]

[3] `https://javaee.github.io/wadl/`.

Management of requests and responses

- Methods with annotations:
  - They can return `void`, a Java type, or `javax.ws.rs.core.Response`
  - If they return a `Response`, they can use the *builder* pattern with the `ResposeBuilder` class
  - If you want to change the HTTP *headers*, you have to return a `Response`
  - Converting between Java types and responses is done with two classes:
    - `MessageBodyReader` for PUT and POST (read)
    - `MessageBodyWriter` for GET (generator)
    - If you do not specify an automatic conversion (we will see later)
  - The Java type returned must have a constructor with a `String`

Parameters of requests

- The annotation `@QueryParam` allows to extract information from the URL query:

  `http://xxx/recurso?step=3&min-m=xxx...`

- Additional annotations: `@DefaultValue`:

```
@Path("smooth")
@GET
public Response smooth(
    @DefaultValue("2") @QueryParam("step") int step,
    @DefaultValue("true") @QueryParam("min-m")
        boolean hasMin,
    @DefaultValue("true") @QueryParam("max-m")
        boolean hasMax,
    ...
```

- `@QueryParam` classes may be standard Java classes, or all those with **a constructor that accepts a** `String`
- Also `@MatrixParam`, `@HeaderParam`, `@CookieParam` and `@FormParam`

Context information

- If you need to obtain a map of associations request parameters (*query*) and strings, you can use the annotation `@Context`:

```
1  @GET
   public String get(@Context UriInfo ui) {
3    MultivaluedMap<String, String> queryParams =
       ui.getQueryParameters();
5    MultivaluedMap<String, String> pathParams =
       ui.getPathParameters();
7  }
```

- And also the HTTP headers:

```
1  @GET
   public String get(@Context HttpHeaders hh) {
3    MultivaluedMap<String, String> headerParams =
       hh.getRequestHeaders();
5    Map<String, Cookie> pathParams =
       hh.getCookies();
7  }
```

Support for producing XML

- It supports serialization and deserialization through *JAXB beans*
  - The class that will be serialized as an XML or JSON resource must be annotated with `@XmlRootElement`
  - If some element of a resource that returns an element of this class and is labeled with the types `@Produces({MediaType.TEXT_XML,Mediatype.APPLICATION_JSON})`, it will be automatically serialized
  - See `http://www.vogella.de/articles/JAXB/article.html`

Support for producing XML

- Serialized class: `Item`

```
@XmlRootElement
public class Item {
    private String uri;
    public void setUri(String uri) {
        this.uri = uri;
    }
    public String getUri() { return uri; }
}
```

- Resource (`/rest/hello/item`):

```
@GET @Path("item")
@Produces(MediaType.TEXT_XML)
public Item item()
{
    Item i = new Item();
    i.setUri("abc");
    return i;
}
```

Support for reading XML

- Resource (`/rest/hello/item`):
  - Its input is a resource directly serializable/deserializable (`item`)

```
@POST
@Consumes(MediaType.TEXT_XML)
@Path("item")
public void newItem(Item i)
{
    System.out.println("Recibido item: " + i.getUri())
        ;
}
```

JSON Representations

- Apart from automatic JSON generation by the Jersey server, JSON can be generated using Java
- Manual serialization to and from JSON is done as shown in `SerializeTest.java` in project `es.um.sisdist.RestTst`
- As an example of serialization:

```
StringWriter writer = new StringWriter();
JAXBContext context = JAXBContext.newInstance(Item
    .class);
Marshaller m = context.createMarshaller();

// Create an Item
Item item = new Item();
item.setUri("http://TestURI");
// Serialize
m.marshal(item, writer);
// Print
System.out.println(writer.toString());
```

JSON Representations II

- The inverse part, deserialization:

```
Unmarshaller um = context.createUnmarshaller();
Item q = (Item)um.unmarshal(new StringReader(
    writer.toString()));
System.out.println(q.getUri());
```

Response.ResponseBuilder

- Used to construct `Response` objects.
- `Response` also has static methods that return `Response.ResponseBuilder`:
  - `ok()` – Correct Answer
  - `created()` – Response to a resource created
  - `status(int status)` – Reply with state
  - `temporaryRedirect(java.net.URI location)` – Redirection to another location specified as a `URI`
  - ...
- More information on the class page: `https://jakarta.ee/specifications/platform/8/apidocs/javax/ws/rs/core/response.responsebuilder`

Response.ResponseBuilder (ii)

- Key Operations
  - `build()` – Build a `Response` from a `ResponseBuilder`
  - `cacheControl()` – Controls cache header
  - `header(java.lang.String name, java.lang.Object value)` – Add a header explicitly
  - `status(int status)` – State
  - `type(MediaType type)` – Add the `Media-type:`
  - ...
- Class page: `https://jakarta.ee/specifications/platform/8/apidocs/javax/ws/rs/core/response`

Media Types and URIs

- `jakarta.ws.rs.core.MediaType`:
  - `TEXT_PLAIN`, `TEXT_HTML`, etc., predefined
  - `MediaType(String type, String subtype)` – Build a *media type* from a type/subtype
  - `boolean isCompatible(MediaType other)` – Compatibility Check
  - `https://jakarta.ee/specifications/platform/8/apidocs/javax/ws/rs/core/mediatype`
- `jakarta.ws.rs.core.UriBuilder` – Builds a `java.net.URI`
  - `fromUri(String uri)` – From String
  - `fragment(String)`, `queryParam(String name, Object[] values)`, `matrixParam(String name, Object[] values)` – Parts of the URI
  - `https://jakarta.ee/specifications/platform/8/apidocs/javax/ws/rs/core/uribuilder`

Using curl to test services

curl:

- -X *method* – Specifies the HTTP method
  - P. ej. -X GET
- -H *header* – Specifies headers to send
  - P. ej. -H Accept:text/xml
- --data *datos* – POST Data
  - P. ej. --data '<?xml><item><uri>...</uri></item>'

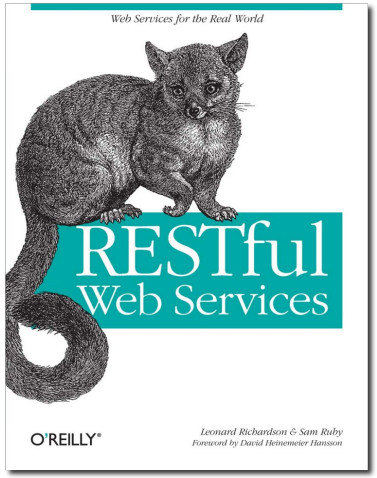
```
1 $ curl -X GET -HAccept:text/xml \
    http://localhost:8080/es.um.sisdist.RestTest/jaxrs/
        hello/item

  <?xml version="1.0" encoding="UTF-8"standalone="yes"?>
2 <item><uri>abc</uri></item>
```

Using curl to test services (ii)

```
$ curl -X POST -HContent-type:text/xml \
  --data '<item><uri>abcdef</uri></item>' \
  http://localhost:8080/es.um.sisdist.RestTest/jaxrs/
      hello/item
```

```
Recibido item: abcdef
```

References



L. Richardson, S. Ruby
RESTful Web Services
O'Reilly, 2007

L. Vogel
RESTful Webservices with Java and Jersey (JAX-RS)
http://www.vogella.de/articles/REST/article.html

Sun Microsystems (Oracle)
RESTful Web Services Developer's Guide
http://docs.sun.com/app/docs/doc/820-4867?l=en&a=load
http://wikis.sun.com/display/Jersey/Overview+of+JAX-RS+1.0+Features

M. Paternostro, K. Hussey
Building RESTful Java Applications with EMF
http://www.slideshare.net/kenn.hussey/building-restful-java-applications-with-emf